Reconnaissance de Chiffres MNIST avec SVM

Classification Multi-classe One-vs-One avec Noyau Polynomial

PRINCE MABENGUE BEBONSU JUNIOR (22T2832)
ETONDE MABONGO ULRICH EDOUARD (22U2163)
FONAYEN CHRISTIAN ASANGWA (23V2370)

Université de Yaoundé I
Département d'Informatique
INF372 - Apprentissage Supervisé et Non Supervisé

Avril 2026

Plan de Présentation

1. Description du Dataset MNIST
2. Prétraitement des Données
3. Théorie et Implémentation du SVM
4. Formulation Mathématique et Résolution QP
5. Entraînement du Modèle
6. Évaluation et Résultats
7. Avantages et Limitations
8. Conclusions et Perspectives

Présentation du Dataset

MNIST - Reconnaissance de chiffres manuscrits

Dimensions :

- Entraînement : 60,000 images — Test : 10,000 images
- Utilisés : 4,000 samples (entraînement et test)
- Format : 28×28 pixels $\rightarrow$ 784 features + 1 label
- Valeurs : pixels 0-255 (niveaux de gris)

Objectif : Classer les 10 chiffres (0-9) avec haute précision

Chargement des Données

Code de chargement :

```
import pandas as pd
import numpy as np

# Chargement des fichiers CSV
df_train = pd.read_csv('mnist_train.csv')
df_test = pd.read_csv('mnist_test.csv')

# S paration features et labels
X_train = df_train.iloc[:, 1:].values  # pixels
y_train = df_train.iloc[:, 0].values   # labels
X_test = df_test.iloc[:, 1:].values
y_test = df_test.iloc[:, 0].values

print(f"X_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"Classes: {np.unique(y_train)}")
```

Vérification de la structure :

- Shape de X_train : (60000, 784)
- Shape de y_train : (60000,)
- Contenu y_train : [5, 0, 4, ..., 5, 6, 8]
- Classes uniques : 0-9

Prétraitement

Étapes :

1. **Normalisation** : $X_{norm} = X/255 \rightarrow [0,1]$
   - Stabilité numérique SVM
   - Évite dominance de features
2. **Sous-échantillonnage** : 4000 samples
   - Entraînement plus rapide

Code de Prétraitement

```
# Normalisation des donn es
X_train = X_train / 255.0
X_test = X_test / 255.0

# Sous-  chantillonnage   pour exp rimentation rapide
sampling = 4000
X_train_sample = X_train[:sampling]
y_train_sample = y_train[:sampling]
X_test_sample = X_test[:sampling]
y_test_sample = y_test[:sampling]

print(f"Training set: {X_train_sample.shape}")
print(f"Test set: {X_test_sample.shape}")
print(f"Valeur min/max: {X_train_sample.min():.2f} / "
      f"{X_train_sample.max():.2f}")
```

Après normalisation :

- Valeurs dans [0.0, 1.0]
- Moyenne 0.13, Écart-type 0.31
- Distribution uniforme entre classes

Support Vector Machine

SVM - Classifieur basé sur la maximisation de marge

Principe : Cherche hyperplan optimal séparant les classes

Formulation :

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

Noyaux : Projettent données en dimension supérieure

- Linear, Polynomial, Sigmoid, RBF

Noyaux Utilisés

Formules :

$$\text{Linear} : K(x, y) = x^T y \tag{1}$$

$$\text{Polynomial} : K(x, y) = (x^T y + b)^d \tag{2}$$

$$\text{Sigmoid} : K(x, y) = \tanh(\gamma x^T y + b) \tag{3}$$

Noyau Polynomial (Deg=2)

$$K(x, y) = (x^T y + 1)^2$$

Paramètres : degree=2, bias=1.0, C=1.0
Avantage : Équilibre expressivité vs complexité

One-vs-One Multi-classe

Nombre de SVMs : $\binom{10}{2} = 45$ paires

Entraînement : Chaque SVM sépare une paire (i, j)

Prédiction : Vote majoritaire sur 45 SVMs

Implémentation One-vs-One

```
for i, class_i in enumerate(classes_):
    for class_j in classes_[i+1:]:
        mask = (y == class_i) | (y == class_j)
        svm = SVM(kernel, C, degree, bias)
        svm.fit(X[mask], y[mask])
        classifiers_[(class_i, class_j)] = svm
```

Prédiction - Vote Majoritaire

```
for (class_i, class_j), svm in classifiers_:
    decision = svm.decision_function(x)
    if decision > 0:
        votes[class_i] += 1
    else:
        votes[class_j] += 1
prediction = max(votes, key=votes.get)
```

Classe gagnante = la plus de votes

Problème d'Optimisation Dual du SVM

Formulation duale du problème SVM :

$$\max_{\lambda} \sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i,j=1}^{n} \lambda_i \lambda_j y_i y_j K(x_i, x_j)$$

Contraintes :

- $\sum_{i=1}^{n} \lambda_i y_i = 0$ (contrainte d'égalité)
- $0 \leq \lambda_i \leq C$ (boîtes)

Problème de Programmation Quadratique (QP) :

$$\min_{\lambda} \frac{1}{2} \lambda^T K \lambda - \mathbf{1}^T \lambda$$

où K est la matrice de Gram : $K_{ij} = y_i y_j K(x_i, x_j)$

Solution : Les λ_i sont les coefficients Lagrange

- $\lambda_i > 0 \rightarrow x_i$ est vecteur de support
- $\lambda_i = 0 \rightarrow x_i$ n'affecte pas la décision

Résolution avec cvxopt

Librairie cvxopt pour résoudre le QP :

```
import cvxopt

# Construction de la matrice de Gram
K = np.dot(X * y[:, None], (X * y[:, None]).T)
P = cvxopt.matrix(K)
q = cvxopt.matrix(-np.ones(n_samples))

# Contraintes d'in galit  : 0 <= lambda_i <= C
G_upper = np.eye(n_samples)
h_upper = np.ones(n_samples) * C
G_lower = -np.eye(n_samples)
h_lower = np.zeros(n_samples)
G = cvxopt.matrix(np.vstack((G_upper, G_lower)))
h = cvxopt.matrix(np.hstack((h_upper, h_lower)))

# Contrainte d'  galit   : sum(lambda_i * y_i) = 0
A = cvxopt.matrix(y.reshape(1, -1), tc='d')
b = cvxopt.matrix(0.0)

# R solution
cvxopt.solvers.options['show_progress'] = False
solution = cvxopt.solvers.qp(P, q, G, h, A, b)
lambda_coeffs = np.array(solution['x']).flatten()
```

Configuration et Entraînement

Paramètres d'entraînement :

```
# Configuration du mod le
model = OneVsOneMultiClassSVM(
    kernel='polynomial',  # Noyau polynomial
    degree=2,             # Degr  du polyn me
    C=1.0,                # Param tre de r gularisation
    bias=1.0              # Biais du noyau
)

# Entra nement sur 4000 samples
sampling = 4000
X_train_sample = X_train[:sampling] / 255.0
y_train_sample = y_train[:sampling]

print("Entra nement en cours...")
model.fit(X_train_sample, y_train_sample)
print("Entra nement termin !")
print(f"Nombre de classificateurs : {len(model.classifiers_)}")
```

Processus d'entraînement :

1. Créer 45 paires de classes (0-1, 0-2, ..., 8-9)
2. Pour chaque paire : entraîner SVM binaire via résolution QP
3. Identifier et stocker vecteurs de support pour chaque SVM
4. Temps total : quelques minutes selon hardware

Métriques

Formules :

$$\text{Accuracy} = \frac{\text{Correctes}}{\text{Total}} \quad \text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{P \times R}{P + R}$$

Résultats Globaux

Performance globale du modèle SVM :

$$\boxed{\text{Accuracy} = \mathbf{0.9295} \approx \mathbf{93\%}}$$

Interprétation :

- **3718 images** correctement classifiées sur 4000
- **282 images** mal classifiées (7.05%)
- Performance **très bonne** pour une méthode SVM classique
- Meilleure que les algorithmes simples (k-NN, régression logistique)
- Inférieure aux réseaux de neurones profonds (CNN : 99

Moyennes globales :

- Macro Average (non-pondérée) : Precision 0.93, Recall 0.93, F1 0.93
- Weighted Average (pondérée) : Precision 0.93, Recall 0.93, F1 0.93
- Support total : 4000 échantillons

Performances Détaillées par Classe

Classe	Precision	Recall	F1-Score	Support
0	0.94	0.97	0.96	370
1	0.94	0.98	0.96	450
2	0.94	0.93	0.93	418
3	0.92	0.90	0.91	408
4	0.92	0.94	0.93	418
5	0.92	0.93	0.92	372
6	0.94	0.94	0.94	378
7	0.92	0.90	0.91	411
8	0.96	0.89	0.92	384
9	0.90	0.91	0.91	391
Moyenne	**0.93**	**0.93**	**0.93**	**4000**

Observations clés :

- Toutes les classes : Precision 0.90, Recall 0.89
- Classe 1 : meilleur Recall (0.98)
- Classe 8 : meilleure Precision (0.96)
- Classes 3, 5, 7 : Recall légèrement plus faible (0.90)

Confusions Principales

- **3 5** : Similarité visuelle (boucles)
- **4** $\rightarrow$ **9** : Variation d'écriture manuscrite
- **7** : Traits inclinés ambigus

Code d'Évaluation

```python
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix

# Pr dictions sur le test set
X_test_sample = X_test[:sampling] / 255.0
y_test_sample = y_test[:sampling]
y_pred = model.predict(X_test_sample)

# Accuracy
acc = accuracy_score(y_test_sample, y_pred)
print(f"Accuracy: {acc:.4f}")

# Rapport de classification
print(classification_report(y_test_sample, y_pred))

# Matrice de confusion
cm = confusion_matrix(y_test_sample, y_pred)
print("Confusion Matrix shape:", cm.shape)  # (10, 10)
```

Visualisation de la matrice de confusion :

- Version brute : valeurs absolues (nombre d'images)
- Version normalisée : pourcentages par vraie classe
- Diagonal = bonnes prédictions
- Hors-diagonal = erreurs

Avantages du SVM pour MNIST

Forces du Support Vector Machine :

1. **Efficacité en haute dimension**
   - 784 features : aucun problème pour SVM
   - Pas de malédiction de dimensionnalité comme pour k-NN
2. **Fondations théoriques solides**
   - Basé sur la théorie de Vapnik-Chervonenkis
   - Bornes de généralisation prouvées mathématiquement
   - Principe du maximum de marge justifié théoriquement
3. **Flexibilité des noyaux**
   - Adaptation à différents types de données
   - Mapping implicite en haute dimension (pas d'explosion computationnelle)
4. **One-vs-One scalable**
   - Utilise sous-ensembles réduits (2 classes par SVM)
   - Parallélisable : 45 SVMs entraînés indépendamment
   - Moins sensible aux données déséquilibrées
5. **Performance 93% sans deep learning**
   - Résultat solide pour approche classique
   - Entraînement rapide comparé aux CNNs

Limitations et Points d'Amélioration

Défis et limitations du SVM :

1. **Temps d'entraînement**
   - 45 SVMs à entraîner (One-vs-One)
   - Résolution QP pour chaque SVM
   - Peut être coûteux pour datasets très larges
2. **Sensibilité aux hyperparamètres**
   - C : balance entre marge et erreurs d'entraînement
   - kernel : choix critique (linear, polynomial, RBF, sigmoid)
   - degree, gamma, bias : paramètres du noyau
   - Nécessite validation croisée/grid search
3. **Choix du noyau**
   - Polynomial deg=2 : ne capture pas toute la complexité
   - RBF : meilleure expressivité, mais plus coûteux
   - Optimal : à déterminer par expérimentation
4. **Interprétabilité limitée**
   - Difficile de comprendre les décisions ("boîte noire")
   - Contrairement aux arbres de décision
5. **Performance plafonnée à 93%**
   - Deep Learning (CNN) : 99.5

Résumé

- SVM binaire + One-vs-One (45 SVMs)
- Noyau polynomial deg=2
- MNIST 60k train, 10k test
- Accuracy : **92.95%**
- All metrics : 0.93 (Precision, Recall, F1)

Perspectives et Améliorations Futures

Directions possibles pour améliorer le modèle :

1. **Optimisation des hyperparamètres**
   - Grid search : C [0.1, 1, 10], degree [1, 2, 3]
   - Validation croisée 5-fold
   - Objectif : trouver combo optimale
2. **Essai de différents noyaux**
   - RBF (Radial Basis Function) : meilleur pour MNIST
   - Combinaison de noyaux
   - Noyaux personnalisés basés sur domain knowledge
3. **Utilisation du dataset complet**
   - Actuellement : 4000 samples d'entraînement
   - Testable : 60000 samples complets
   - Peut améliorer généralisation (mais temps plus long)
4. **Comparaison avec Deep Learning**
   - CNNs simples : 99.5
   - Trade-off : SVM moins gourmand (pas GPU)
   - Ensemble methods : combiner SVM + autres classifieurs
5. **Data augmentation**
   - Rotations légères, bruits ajoutés

Références

- Vapnik (1995) - SVM Theory
- Bishop (2006) - ML Textbook
- LeCun et al. - MNIST Dataset
- scikit-learn, cvxopt

Merci de votre attention !

Questions ?

Groupe 1 : PRINCE MABENGUE (22T2832), ETONDE MABONGO (22U2163), FONAYEN CHRISTIAN (23V2370)